Лабораторная работа №7 "Создание компонентного интерфейса с использованием React"

Лабораторная работа №7. Создание компонентного интерфейса с использованием React

1. ЦЕЛЬ РАБОТЫ

Освоение концепций компонентного подхода в React. Практическое знакомство с созданием компонентов, работой с JSX и передачей данных через props. Подготовка компонентной базы для последующего управления состоянием и интеграции с сервером (во 2 части курса, следующий семестр).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Продвинутый JavaScript (ES6+: стрелочные функции, деструктуризация, модули, spread/rest).
- HTML5 и CSS3 (семантическая вёрстка, базовые стили, адаптивность).
- Основы компонентного подхода в веб-разработке.
- Работу с API (из лабораторной работы №6) – как база для будущей интеграции.

---

3. ЗАДАЧИ РАБОТЫ

- Создать React-приложение с использованием Vite.
- Изучить синтаксис JSX и его отличия от обычного HTML.
- Научиться создавать функциональные компоненты.
- Освоить передачу данных между компонентами через `props`.
- Понять концепции SPA, Virtual DOM и различия между фреймворками (React, Vue, Angular).
- Создать библиотеку переиспользуемых компонентов для дальнейшего использования в курсовом проекте.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ (общий для всех вариантов)

Шаг 1. Подготовка рабочего окружения

1. **Откройте терминал** и перейдите в папку ваших проектов.
2. **Создайте новое React-приложение с помощью Vite:**

```
npm create vite@latest my-react-app -- --template react
```

3. **Перейдите в созданную папку проекта** и установите зависимости:

```
cd my-react-app
npm install
```

4. **Установите React Router** (для будущих работ по маршрутизации, но в этой работе не используется):

```
npm install react-router-dom
```

5. **Запустите приложение в режиме разработки:**

```
npm run dev
```

   Приложение будет доступно по адресу `http://localhost:5173`.

Шаг 2. Знакомство со структурой проекта и организация файлов

Создайте следующую структуру папок в `src/` (в соответствии с вашей тематикой курсового проекта):

```
src/
├── components/
│   ├── ui/           - базовые UI-компоненты (Button, Card, Badge и т.п.)
│   ├── layout/       - компоненты макета (Header, Footer, Sidebar)
│   └── features/     - предметные компоненты (ProductCard, NewsItem, TaskItem и т.п.)
├── pages/            - (заготовка для будущей маршрутизации)
├── data/             - моковые данные (массивы объектов)
├── App.jsx           - корневой компонент
├── main.jsx          - точка входа
└── index.css         - глобальные стили (можно использовать CSS-модули или styled-components)
```

Шаг 3. Создание базовых UI-компонентов

Пример компонента кнопки (`src/components/ui/Button.jsx`):

```
function Button({ children, onClick, variant = 'primary', ...props }) {
    const className = `btn btn--${variant}`;
    return (
        <button className={className} onClick={onClick} {...props}>
            {children}
```

```
        </button>
    );
}
export default Button;
```

Пример компонента карточки (`src/components/ui/Card.jsx`):

```
function Card({ children, title, className = '' }) {
    return (
        <div className={`card ${className}`}>
            {title && <div className="card__title">{title}</div>}
            <div className="card__content">{children}</div>
        </div>
    );
}
export default Card;
```

Шаг 4. Создание моковых данных

В папке `src/data/` создайте файл, соответствующий вашей тематике (например, `products.js`, `news.js`, `tasks.js`):

```
export const mockProducts = [
    { id: 1, name: 'Смартфон', price: 29990, rating: 4.5, image: '...', category: 'electronics' },
    { id: 2, name: 'Ноутбук', price: 59990, rating: 4.8, image: '...', category: 'electronics' }
];
```

Шаг 5. Создание композитных (предметных) компонентов

Пример компонента карточки товара (`src/components/features/ProductCard.jsx`):

```
import Card from '../ui/Card';
import Button from '../ui/Button';

function ProductCard({ product, onAddToCart }) {
    return (
        <Card title={product.name}>
            <img src={product.image} alt={product.name} />
            <p>Цена: {product.price} ₽</p>
            <p> {product.rating}</p>
            <Button onClick={() => onAddToCart(product)}>В корзину</Button>
        </Card>
    );
}
export default ProductCard;
```

Шаг 6. Компонент списка с фильтрацией (опционально, но рекомендуется)

Пример компонента `ProductList.jsx`:

```
import ProductCard from '../features/ProductCard';
import { useState } from 'react'; // но без useEffect (по заданию)
function ProductList({ products, onAddToCart }) {
    const [category, setCategory] = useState('all');
    const filtered = category === 'all' ? products : products.filter(p => p.category === category);
    return (
        <div>
            <select onChange={(e) => setCategory(e.target.value)} value={category}>
                <option value="all">Все</option>
                <option value="electronics">Электроника</option>
                {/* другие категории */}
            </select>
            <div className="product-grid">
                {filtered.map(product => (
                    <ProductCard key={product.id} product={product} onAddToCart={onAddToCart} />
                ))}
            </div>
        </div>
    );
}
export default ProductList;
```

Шаг 7. Главный компонент приложения (`App.jsx`)

```
import Header from './components/layout/Header';
import ProductList from './components/features/ProductList';
import { mockProducts } from './data/products';

function App() {
    const handleAddToCart = (product) => {
        console.log('Добавлен в корзину:', product);
        // в следующих работах будет реализовано состояние
    };
    return (
        <div>
            <Header title="Магазин" />
            <main>
                <ProductList products={mockProducts} onAddToCart={handleAddToCart} />
            </main>
        </div>
    );
}
export default App;
```

Шаг 8. Стилизация (CSS-модули или обычные CSS)

Создайте файлы стилей (например, `Button.module.css`, `ProductCard.css`) и подключите их в компонентах.

---

5. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ (выбор темы)

Каждый студент работает над своим проектом. Индивидуальное выполнение. Все выполняют единый порядок действий (задание одинаково для всех вариантов, отличается только тематика проекта, которая выбирается в соответствии с будущим курсовым проектом).

- **!!!! Для выбора темы используется Генератор вариантов**

Общее задание для всех вариантов:

Создайте React-приложение, соответствующее вашей предметной области, с соблюдением следующих требований:

- **Минимум 5 различных компонентов** (включая базовые UI и предметные).
- **Композитные компоненты**, объединяющие более простые.
- **Моковые данные** (массивы объектов) – не менее 5–10 записей.
- **Обработчики событий** (заглушки) – например, на кнопках «добавить», «удалить», «фильтр» (без изменения состояния, просто `console.log`).
- **Стилизация компонентов** (CSS, CSS-модули или styled-components).
- **Использование `props`** для передачи данных от родителя к дочерним компонентам.
- **Запрещено использовать `useState` и `useEffect`** (все данные – статические моковые, состояние не хранится).

Рекомендации по составу компонентов в зависимости от предметной области:

Тип проекта	Примеры компонентов
Интернет-магазин	`Header`, `ProductCard`, `ProductList`, `CartItem`, `PriceFilter`
Новостной портал	`Header`, `NewsCard`, `NewsList`, `CategoryTabs`, `SearchBox`
Блог / платформа курсов	`PostCard`, `AuthorCard`, `TagList`, `CommentPreview`, `RatingStars`
Доска задач (TODO)	`TaskItem`, `TaskList`, `AddTaskForm`, `PriorityBadge`, `FilterButtons`
Каталог фильмов	`MovieCard`, `GenreFilter`, `RatingStars`, `SearchBox`, `MovieGrid`
Погодное приложение	`WeatherCard`, `CitySearch`, `ForecastList`, `Temperature`, `WeatherIcon`

Вы можете использовать любую другую тему, близкую к вашему курсовому проекту.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- **Индивидуальное выполнение.**
- Код должен быть загружен в репозиторий GitHub (ветка `feature/react-components`).
- В README репозитория укажите:

- Предметную область проекта.
- Инструкцию по запуску (`npm install`, `npm run dev`).
- Список созданных компонентов и их назначение.

---

7. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист (по Стандарту Предприятия).
- Цель работы.
- Краткие теоретические сведения:
  - Компонентный подход в React.
  - Синтаксис JSX и его отличия от HTML.
  - Концепция SPA и Virtual DOM.
  - Сравнение фреймворков (React vs Vue vs Angular – по выбору).
- **Ход работы:**
  - Описание созданных компонентов, их иерархии и назначения.
  - Примеры использования JSX и `props` (код из проекта).
  - Скриншоты работающего приложения (на разных разрешениях экрана).
  - Исходный код ключевых компонентов (или ссылка на репозиторий).
- **Ответы на контрольные вопросы**.
- **Выводы** (что узнали, какие навыки приобрели, как подготовили основу для курсового проекта).

---

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое JSX и чем он отличается от обычного HTML? Приведите 3 конкретных примера различий.
2. Как работает передача данных между компонентами через `props`? Покажите пример из вашего проекта.
3. В чём преимущества компонентного подхода по сравнению с традиционной разработкой на чистом JavaScript?
4. Объясните концепцию Virtual DOM и её преимущества для производительности веб-приложений.

5. Сравните React, Vue и Angular по трём критериям: кривая обучения, производительность, экосистема.
6. Почему в React важно, чтобы компоненты были чистыми функциями (относительно props)?
7. Как в JSX вставить JavaScript-выражение? Приведите пример из вашего кода.
8. В чём разница между `props` и `state`? Почему в этой лабораторной работе мы не использовали `state`?
9. Как организовать условный рендеринг в JSX? Покажите пример из вашего компонента.
10. Как отрендерить список элементов в React? Зачем нужен атрибут `key`?
11. Что такое «подъём состояния» (lifting state up) и когда он применяется? (теоретически)
12. Какие типы данных могут быть переданы через `props`? Приведите примеры из вашего проекта.
13. Как организовать композицию компонентов (один компонент внутри другого) с использованием `children`?
14. Что такое фрагменты (`<></>`) и зачем они нужны?
15. Как в React обрабатывать события (click, change) без использования состояния? (просто вызов функции-заглушки).
16. Как настроить CSS-модули в проекте, созданном с Vite?
17. Какие преимущества даёт использование Vite по сравнению с Create React App?
18. Как в вашем проекте организована передача моковых данных – через `import` в компоненты или через `props`?
19. Какие компоненты в вашем проекте являются универсальными (UI), а какие – предметными (features)? Почему вы их разделили?
20. Как вы планируете развивать этот проект в следующих лабораторных работах (добавление состояния, маршрутизация, работа с сервером)?

---

9. КРИТЕРИИ ОЦЕНКИ

- Корректная настройка React-приложения с Vite и структурирование проекта.
- Создание не менее 5 функциональных компонентов (в том числе переиспользуемые UI-компоненты).
- Использование `props` для передачи данных между компонентами.
- Наличие моковых данных (массив объектов) и корректный рендеринг списка.
- Отсутствие использования `useState` и `useEffect` (данные только статические).
- Соответствие тематики проекта будущему курсовому проекту.

- Стилизация компонентов (CSS-модули или обычный CSS).
- Полнота отчёта (скриншоты, код, ответы на вопросы).
- Своевременная сдача и наличие рабочего репозитория.

---

10. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Эта лабораторная работа является **подготовительной** для всего цикла React: в следующем семестре вы добавите управление состоянием (useState/useEffect), маршрутизацию (React Router) и взаимодействие с сервером (fetch/axios).
- Все данные сейчас – статические моковые; это позволяет сосредоточиться на компонентах и `props`, не отвлекаясь на асинхронность.